

Procedure

LAB-1

1. Download and install Eclipse IDE.
2. Install JDK and set `JAVA_HOME` and `PATH` variables.
3. Download Maven ZIP file from Apache Maven website.
4. Extract Maven folder and add Maven `bin` path to Environment Variables.
5. Open Command Prompt and check Maven installation using:

<> Bash



```
mvn -v
```

6. Download Gradle ZIP file.
7. Extract Gradle files into `C:\Gradle`.
8. Add Gradle `bin` path to Environment Variables.
9. Open Command Prompt and verify Gradle installation using:

<> Bash



```
gradle -v
```



Procedure

LAB-2

1. Open Eclipse IDE.
2. Go to:

File → New → Maven Project



3. Select **Use default Workspace Location** and click **Next**.
4. Choose:

maven-archetype-quickstart



5. Enter the following details:

Group Id : `com.program2.maven`

Artifact Id : `program2-example-jar`

Package : `com.program2.maven.program2`



6. Click **Finish** to create the Maven project.
7. Right-click on the project and select:

Maven Build



8. Enter Goal as:

package



9. Click **Run** to build the project.
10. Open `App.java` and run the Java application.

Procedure

LAB-3

1. Open Command Prompt.
2. Create a new directory:

```
</> Bash
mkdir pgm3
```



3. Move to the directory:

```
</> Bash
cd pgm3
```



4. Initialize Gradle project:

```
</> Bash
gradle init
```



5. Select the following options:

- Application type
- Groovy as implementation language
- Java version
- Project name
- Single application structure
- Kotlin DSL

6. Wait until the message **BUILD SUCCESSFUL** appears.

7. Run the Gradle project using:

```
</> Bash
gradlew run
```



8. To view project structure, use:

```
</> Bash
tree
```



Procedure

LAB-4

1. Create a Maven project as done in Program 2.
2. Build and run the Java application to get the **Hello World** output.
3. Open terminal in Eclipse using:

```
Ctrl + Alt + Shift + T
```



4. Run the following command:

```
</> Bash
gradle init
```



5. Select **Yes** for migration from Maven to Gradle.
6. Choose **Kotlin DSL** option.
7. Validate prompts and complete initialization.
8. Build the Gradle project using:

```
</> Bash
gradle build
```



9. Update the `build.gradle` file if required.
10. Run the following commands:

```
</> Bash
gradle clean build
gradle run
```



11. Verify the output message:

```
Hello World! Welcome to pgm4
```



Procedure

LAB-5

1. Download Jenkins for Windows from the Jenkins official website.
2. Install Jenkins using the MSI installer.
3. Select **Run service as Local System** during installation.
4. Choose port number as:

8080

5. Verify Java installation using:

<> Bash

```
java -version
```

6. Complete the Jenkins installation process and click **Finish**.
7. Open browser and launch Jenkins using:

http://localhost:8080/

8. Copy the administrator password from:

C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword

9. Paste the password in Jenkins setup page.
10. Select **Install Suggested Plugins**.
11. Complete the basic configuration and open Jenkins dashboard.

Procedure

LAB-6

1. Open Jenkins Dashboard.
2. Click on:

Manage Jenkins → Plugins

3. Search and install **Maven Integration Plugin**.
4. Go to:

Manage Jenkins → Tools

5. Configure Maven installation path (`MAVEN_HOME`).
6. Click on **New Item**.
7. Enter project name and select:

Freestyle Project

8. Under **Build**, click:

Add Build Step → Invoke top-level Maven targets

9. Select Maven version and enter goal:

clean install

10. Add the `pom.xml` file path in Advanced settings.
11. Click **Save** and then **Build Now**.
12. View build output in **Console Output**.

LAB-7

1. Create a new user using:

```
</> Bash  
sudo adduser username
```

2. Give sudo privileges:

```
</> Bash  
sudo usermod -aG sudo username
```

3. Switch to the new user:

```
</> Bash  
sudo su username
```

4. Generate SSH key:

```
</> Bash  
ssh-keygen
```

5. Copy SSH key to remote host:

```
</> Bash  
ssh-copy-id username@remote-host
```

6. Update Ubuntu packages:

```
</> Bash  
sudo apt update
```

7. Install Ansible:

```
</> Bash  
sudo apt install ansible -y
```

8. Verify installation:

```
</> Bash  
ansible --version
```

9. Create inventory directory:

```
</> Bash
```

10. Create hosts file:

```
</> Bash  
sudo nano /etc/ansible/hosts
```

11. Add localhost entry in hosts file:

```
[local]  
localhost ansible_connection=local
```

12. Check inventory:

```
</> Bash  
ansible-inventory --list -y
```

13. Test connection using:

```
</> Bash  
sudo ansible all -m ping
```